

ReadAlert – Delivery 1

Wayne has provided an example skeleton/structure of how a working module looks like. The file can be found at: [Cyber-Safety-Centre/Cobbled-DevResources at Working-Module.](#)

Download the ZIP from GitHub and open the Cobbled-EDP-3.0.0 folder in Visual Studio 2022.

This is the layout of a working module, as confirmed by Wayne. To start, in the Database folder, start working in the ‘**no-rls**’ folder as it is most suitable for building and debugging. Inside this no-rls folder, we need to create SQL files for each section: **ReaderProfile, Authors, Books, User books, Book Lookup**. Each file should start with table creation, followed by stored operations. Here is an example of what that looks like (taken from the example module, EDP).

Wayne has provided complete stored procedures if you would like to use them, found in the **18. red - Cobble_ReadAlert_Delivery1_SP_API_Pack_v2.0** ZIP folder.

We then need to run this against a real database; Wayne has provided some in the following ZIP:

02. MMS – Member Management System

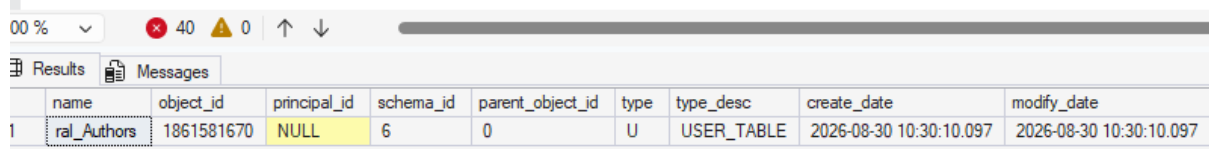
Which has the seed data pack, as seen below.

```
1  # Cobble mms_ Seed Data Pack v1.0
2
3  Target databases:
4  - `Cobbled_NoRLS`
5  - `Cobbled_RLS`
6
7  Run order:
8
9  ## NoRLS
10 1. Run `Cobbled_NoRLS_mms_Module_Delivery1_NoRLS_Build_v1_0.sql`
11 2. Run `Cobbled_NoRLS_mms_Seed_Baseline_v1_0.sql`
12 3. Run `Cobbled_NoRLS_mms_Seed_Verification_v1_0.sql`
13
14 ## RLS
15 1. Run `Cobbled_RLS_mms_Module_Delivery1_RLS_Build_v1_0.sql`
16 2. Run `Cobbled_RLS_mms_Seed_Baseline_v1_0.sql`
17 3. Run `Cobbled_RLS_mms_Seed_Verification_v1_0.sql`
18
19 Rollback scripts are included for both environments.
20
21 Delivery 1 notes:
22 - `mms_Relationship` remains parked/deferred.
23 - Personal identity contact relationships are seeded through `mms_Contact`.
24 - Lookup values should be conformed through `glb`; the catalogue is included as CSV, not forced into this mms_script.
25
```

Open SSMS (SQL Server Management Studio), connect to your local SQL server. Run the mms_seed scripts to test **TenantID** and **MemberID** values.

Then run the SQL file you created against Cobbled_NoRLS. I used the following code to test the procedures:

```
87  /* Testing if table and procedures exist */
88  SELECT * FROM sys.tables WHERE name = 'ral_Authors';
89  SELECT * FROM sys.procedures WHERE name = 'ral_Authors_CRUD_JSON';
```



	name	object_id	principal_id	schema_id	parent_object_id	type	type_desc	create_date	modify_date
1	ral_Authors	1861581670	NULL	6	0	U	USER_TABLE	2026-08-30 10:30:10.097	2026-08-30 10:30:10.097

```

/* Setting connection to a mms user using TenantID and MemberID, and creating a new Author */
EXEC sys.sp_set_session_context @key = N'TenantID', @value = 'F115B0A094467E7FCBA8DB16DF52242E';
EXEC sys.sp_set_session_context @key = N'MemberID', @value = 'AAAAAAAAAAAAAAAAAAAAAAAAAAAA';

DECLARE @Result NVARCHAR(MAX);
EXEC ral_ral_Authors_CRUD_JSON @Action = 'INSERT', @Payload = N'{"AuthorName": "Test Author"}', @Result = @Result OUTPUT;
SELECT @Result;

```

And these are the results:

```

1  {
2      "TenantID": "F115B0A094467E7FCBA8DB16DF52242E",
3      "AuthorID": "135C5A861F8C4857AFC00FA6423396D2",
4      "AuthorName": "Test Author",
5      "IsVerified": false,
6      "CreatedByMemberID": "AAAAAAAAAAAAAAAAAAAAAAAAAAAA",
7      "CreatedUtc": "2026-08-30T01:29:05.8802300",
8      "UpdatedUtc": "2026-08-30T01:29:05.8802300",
9      "IsActive": true
10 }

```

We can then create the Endpoints in Program.cs. As seen in the example module program, Wayne expects a small, minimal API endpoint.

```

// Health stays unauthenticated; MOD and observability use it to distinguish a down
// service from a valid request which the user cannot perform.
app.MapGet("/aum/edp/v3.0/health", () => Results.Ok(new { service = "EDP", version = "3.0.0", status = "healthy" }));
app.MapGet("/aum/edp/v3.0/readiness", async () =>
{
    try { await using var db = new SqlConnection(connectionString); await db.OpenAsync();
        return Convert.ToInt32(await new SqlCommand("SELECT IIF(OBJECT_ID(N'edp.edp_ExampleItem_CRUD_JSON', N'P') IS NULL, 0, 1)", db).Execute
        catch { return Results.StatusCode(503); }
});

// Shell frames this entry point and posts a signed host context to frontend/app.js.
app.MapGet("/aum/edp/{version}/items", (string version, IWebHostEnvironment env) =>
    version.StartsWith("3.0.", StringComparison.Ordinal) ? Results.File(Path.Combine(env.WebRootPath, "edp-widget", "index.html"), "text/ht

app.MapGet("/aum/edp/v3.0/items", async (HttpRequest request) =>
{
    var scope = await ReadScopeAsync(request, app.Configuration);
    if (scope is null) return Results.Unauthorized(); if (!scope.Has("edp.items.read")) return Forbidden("edp.items.read");
    return Json(await ExecuteCrudAsync(connectionString, scope, "SELECT", new { }));
});
app.MapPost("/aum/edp/v3.0/items", async (HttpRequest request, ExampleItemInput input) =>
{
    var scope = await ReadScopeAsync(request, app.Configuration);
    if (scope is null) return Results.Unauthorized(); if (!scope.Has("edp.items.write")) return Forbidden("edp.items.write");
    if (string.IsNullOrEmpty(input.Title)) return Results.BadRequest(new { code = "EDP_TITLE_REQUIRED" });
    return Json(await ExecuteCrudAsync(connectionString, scope, "INSERT", new { title = input.Title.Trim(), notes = input.Notes?.Trim() }));
});
app.MapPut("/aum/edp/v3.0/items/{itemId}", async (HttpRequest request, string itemId, ExampleItemInput input) =>
{
    var scope = await ReadScopeAsync(request, app.Configuration);
    if (scope is null) return Results.Unauthorized(); if (!scope.Has("edp.items.write")) return Forbidden("edp.items.write");
    if (itemId.Length != 32 || string.IsNullOrEmpty(input.Title)) return Results.BadRequest(new { code = "EDP_INPUT_INVALID" });
    return Json(await ExecuteCrudAsync(connectionString, scope, "UPDATE", new { exampleItemId = itemId, title = input.Title.Trim(), notes =

});
app.MapDelete("/aum/edp/v3.0/items/{itemId}", async (HttpRequest request, string itemId) =>
{
    var scope = await ReadScopeAsync(request, app.Configuration);
    if (scope is null) return Results.Unauthorized(); if (!scope.Has("edp.items.delete")) return Forbidden("edp.items.delete");
    return Json(await ExecuteCrudAsync(connectionString, scope, "DELETE", new { exampleItemId = itemId }));
});

```

Wayne has specified the endpoint routes in the **Cobble_ReadAlert_Delivery1_Microservice_API_Contract_v2.0** document and more information about module development in the **05_Cobbled_Module_Package_Developer_Instructions_v3.0** document.